Assignment: Build a Search Typeahead System

1. Overview

In this assignment, students will build a search typeahead system similar to the suggestion feature seen in search engines, e-commerce platforms, and content platforms. The system should suggest popular search queries while the user is typing, support search submissions, update query popularity, and use caching to achieve low-latency reads.

The focus of this assignment is the backend data-system design: how query-count data is stored, how suggestions are served quickly, how cache distribution is handled, and how write pressure is reduced.

2. Problem Statement

Build a working search typeahead application with the following capabilities:

1. When a user types in the search box, the system should show 10 suggestions sorted by search count.
2. The application should include a UI interface for searching and displaying suggestions.
3. The backend should expose a dummy search API that returns a response such as "Searched".
4. Whenever a search is submitted, the search-query data store should be updated.
5. Students should design how query-count data is stored and how caching is used for low latency.
6. The cache layer should be distributed using consistent hashing.
7. The system should support trending searches.
8. The system should support batch writes for search-count updates.

3. Dataset Requirement

Students may use any open-source dataset containing search queries, keywords, product names, page titles, or similar text entries. The dataset should include a count or frequency value for each query. If the chosen dataset does not already include counts, students may derive counts by aggregation.

Expected input format:

query	count
iphone	100000
iphone 15	85000
iphone charger	60000

query	count
java tutorial	40000

Minimum expected dataset size: 100,000 queries. Larger datasets are encouraged.

4. Functional Requirements

4.1 Typeahead Suggestions

Whenever the user types a prefix in the search box, the system should return suggestions matching that prefix.

- Return at most 10 suggestions.
- Suggestions must start with the typed prefix.
- Suggestions must be sorted by count in descending order.
- The system should handle empty input, missing input, mixed-case input, and prefixes with no matches gracefully.
- The UI should avoid unnecessary backend calls, for example by using debouncing.

4.2 Search Submission

When the user submits a search, the backend should return a dummy response and update the query-count data.

- If the query already exists, its count should increase.
- If the query does not exist, it should be inserted with an initial count.
- The dummy search API should return a response such as: `{ "message": "Searched" }`.
- The update should eventually be reflected in suggestions and trending searches.

5. API Expectations

The following APIs are expected at minimum. Students may add more APIs if required.

API	Purpose	Expected Behavior
GET /suggest?q=<prefix>	Fetch suggestions	Returns up to 10 prefix-matching suggestions sorted by count.
POST /search	Submit search	Returns "Searched" and records the submitted query.
GET /cache/debug?prefix=<prefix>	Debug cache routing	Shows which cache node is responsible for the prefix and whether it is a hit or miss.

6. Data Storage and Caching Expectations

Students must decide how to store search-query data and how to serve suggestions with low latency. They are expected to justify their design choices in the submission and during the viva/mock interview.

- The system should maintain query-count data reliably enough for the assignment demo.
- The suggestion flow should use a cache before falling back to the primary data store.
- The cache should store suggestion results for prefixes.
- The cache should support expiry or invalidation so stale data does not remain forever.
- The cache should be distributed across multiple logical cache nodes.
- Consistent hashing must be used to decide which cache node owns a prefix key.

7. Trending Searches

The basic version of the typeahead system (for 60% marks) should return suggestions sorted by the **overall search count**. This means that historically popular queries should appear first.

For the additional **20% marks**, students are expected to improve this ranking by incorporating **recency**. In this version, suggestions should not be sorted only by all-time popularity; instead, recently searched queries should get higher priority.

Students should design a reasonable approach to combine historical popularity and recent activity. The exact scoring formula is left to the students, but they must clearly explain:

1. How recent searches are tracked.
2. How recent activity affects ranking.
3. How the system avoids permanently over-ranking queries that were popular only for a short period.
4. How the cache is updated or invalidated when rankings change.
5. What trade-offs their approach makes between freshness, latency, and implementation complexity.

The expected behavior is that the **same suggestion API** should support this improved ranking.

The core API remains:

- GET /suggest?q=<prefix>

Basic version:

- Sort matching suggestions by overall count.

Enhanced version:

- Sort matching suggestions using a recency-aware ranking mechanism.

Students should demonstrate the difference between the two ranking approaches using sample data or logs.

8. Batch Writes

Students must support batch writes for search-count updates. The goal is to avoid writing to the primary data store synchronously for every search request.

- Search submissions should be collected in a buffer, queue, log, or equivalent mechanism.
- Repeated queries should be aggregated before being written.
- The batch writer should flush periodically or after reaching a configurable batch size.
- Students should show how batch writes reduce the number of database writes.
- Students should discuss the failure trade-offs of their approach, especially what happens if the application crashes before a batch is flushed.

9. UI Requirements

- Search input box.
- Suggestion dropdown that updates as the user types.
- Search submission on pressing Enter or clicking a search button.
- Display of the dummy search response.
- Trending searches section.
- Loading and error states.
- Basic keyboard support for navigating suggestions.
- Clean and usable layout.

10. Non-Functional Expectations

- The system should be easy to run locally.
- The suggestions API should be optimized for low latency.
- Students should measure and report latency, preferably including p95 latency.
- Students should report cache hit rate and database read/write counts where possible.
- Students should include logs or a short explanation showing consistent-hashing behavior.
- The code should be modular, readable, and documented.

11. Use of AI and Academic Integrity

Use of AI tools is allowed for this assignment. However, students are fully responsible for understanding their submission.

After submission, students are expected to be in a position to explain every major design choice and the core implementation code. This includes data modeling, caching, consistent hashing, trending-search computation, batch-write logic, and important code snippets used in the submitted project.

If a student is unable to explain their design choices or core implementation during a viva/mock interview, the submission may be treated as plagiarism, even if the code runs correctly.

12. Expected Submission

- GitHub repository or equivalent source-code submission.
- README with setup instructions.
- Dataset source and loading instructions.
- Architecture diagram or clear architecture explanation.
- API documentation.
- Screenshots or short demo video.
- Performance report covering latency, cache hit rate, and write reduction through batching.
- Explanation of design choices and trade-offs.

13. Grading Rubric: 100 Marks

Component	Marks	Expectation
Basic Implementation	60	Working dataset ingestion, search UI, suggestions API, search API, query-count updates, and distributed cache using consistent hashing.
Trending Searches	20	Clear and working trending-search implementation and explanation of the scoring/windowing logic.
Batch Writes	20	Batching or sampling, write-reduction evidence, and discussion of failure trade-offs.

14. Suggested Milestones

1. Load dataset and build the basic suggestion API.
2. Build the frontend search box and suggestion dropdown.
3. Add dummy search submission and query-count updates.
4. Add distributed cache with consistent hashing.
5. Add trending searches.
6. Add batch writes.
7. Measure performance and prepare final documentation/demo.